

Lokalizációs készségfejlesztő alkalmazás

Készítették:
Eördögh Erik
Pintér Bálint
Varga Norbert

Tartalomjegyzék

1	Bevezetés.....	1
1.1	A téma ismertetése és a program bemutatása	1
1.2	Szakmai és hasznosulási célkitűzések	1
1.3	Témaválasztás indoklása a készítőik szemszögéből.....	1
2	Fejlesztői dokumentáció.....	2
2.1	Bevezetés	2
2.2	Fejlesztőkörnyezet	2
2.2.1	Hardverkörnyezet	2
2.2.2		2
2.2.3	Szoftverkörnyezet.....	2
2.2.4	A technológiai választás indoka	3
2.3	Kialakított adatszerkezetek	3
2.3.1	Adatbázis-séma áttekintése	3
2.3.2	Felhasználói és jogosultsági adatok.....	4
2.3.3	Játéktérkép és képi erőforrások	5
2.3.4	Pontok és kapcsolatok	5
2.3.5	Játékmeneti állapot	6
2.3.6	Kiegészítő adatszerkezetek.....	6
2.3.7	A C++ WebAssembly motor belső adatszerkezetei	6
2.4	Rendszerarchitektúra	8
2.5	A backend részletes bemutatása	8
2.5.1	Projektstruktúra.....	8
2.5.2	REST API végpontok.....	9
2.5.3	Hitelesítés és munkamenet kezelés	12
2.5.4	Képfeldolgozás.....	12
2.6	A frontend bemutatása.....	12
2.7	WebAssembly Engine.....	16
2.7.1	Az Engine felépítése.....	16
2.7.2	A motor feladatai.....	16
2.8	A program tipikus algoritmusai.....	16
2.8.1	Panorámaképek gömbszegmens-alapú renderelése.....	16
2.8.2	Az oldal kétnyelvűsítése (Lokalizáció).....	18
2.8.3	Pontszámítás algoritmusai.....	19
2.8.4	Cikluskezelés: ismétlődésmentes pontkiosztás	19

2.8.5	Navigációs irány számítása	20
2.8.6	Képfeldolgozás menete	20
2.8.7	Törölt felhasználók kezelése és anonimizáció	20
2.9	Teszt dokumentáció	20
2.9.1	Egységtesztelés (Jest)	20
2.9.2	Böngészőtesztelés	22
2.9.3	Validáció és hibakezelés tesztelése	23
2.9.4	Tesztelői hozzáférés.....	23
2.10	Továbbfejlesztési lehetőségek	23
3	Felhasználói dokumentáció	25
3.1	Az alkalmazás célja és főbb funkciói	25
3.2	Szükséges hardver és szoftvereszközök.....	25
3.3	Telepítés és indítás.....	26
3.3.1	WebAssembly buildelése.....	27
3.4	A program részletes bemutatása	28
3.4.1	Regisztráció és bejelentkezés	28
3.4.2	Profilbeállítások.....	29
3.4.3	Pályák böngészése	30
3.4.4	Pályoldal.....	30
3.4.5	Játékmenet	30
3.4.6	Saját pálya készítése	31
3.5	Adminisztrációs felület.....	31
3.6	Hibajelzések	31
4	Csapatmunka megvalósítása	33
5	Összefoglalás	34
5.1	Projekt kezdete	34
5.2	Projekt során.....	34
5.3	Tanulságok és tapasztalatok	34
5.4	A program jövője.....	34
6	Ábrajegyzék	35
7	Irodalomjegyzék	36
7.1	Weboldalak	36

1 Bevezetés

1.1 A téma ismertetése és a program bemutatása

Vizsgaprojektünk egy webes alapú, lokalizációs készségfejlesztő alkalmazás, a Lokea. A felhasználóknak különböző virtuális pályákon – például egy videójáték térképén – kell egy 360 fokos panorámakép alapján meghatározniuk a tartózkodási helyüket, majd azt megjelölniük egy interaktív térképén. A beépített helyszínek mellett a játékosok saját pályákat is létrehozhatnak, a megszerzett pontjaikat pedig egy globális ranglistán mérhetik össze a többi felhasználóval.

1.2 Szakmai és hasznosulási célkitűzések

Szakmai célunk egy többretegű webes alkalmazás létrehozása volt, amely a gyakorlatban is teszteli a szerver- és kliensoldali programozási, illetve az adatbázis-kezelési ismereteinket. A 360 fokos képek renderelését egy saját fejlesztésű C++ alapú motor biztosítja, az együttműködést pedig GitHub segítségével koordináltuk. A program gyakorlati célja, hogy játékos formában fejlessze a téri tájékozódást és a térképolvasási készséget, amely a mindennapos GPS-használat mellett sokszor háttérbe szorul.

1.3 Témaválasztás indoklása a készítők szemszögéből

A projekt alapötletét egy már létező, panorámakép-alapú helymeghatározós játék adta. Ezt a koncepciót szerettük volna továbbfejleszteni olyan egyedi funkciókkal, mint a saját pályák készítésének lehetősége. A téma választása szakmailag azért volt előnyös, mert a projekt moduláris felépítése jól elkülöníthető fejlesztési feladatokra (frontend, felhasználókezelés, C++ játékmotor) bontható, így a csapat tagjai párhuzamosan és hatékonyan tudtak rajta dolgozni.

2 Fejlesztői dokumentáció

2.1 Bevezetés

A vizsgaprojektünk szerveroldala egy Express alapú REST API-ra épül, amely a játékmenet, a felhasználói jogosultságkezelés, a térképszerkesztés, a képfeltöltések és az adminisztrációs funkciók kiszolgálásáért felel.

A projekt szerkezetéből következően három fő technikai réteg különíthető el: a MySQL adatmodell, a Node.js/Express backend, illetve a böngészőben futó kliens és WebAssembly motor. A dokumentáció ennek megfelelően elsősorban a backend és az adatbázis oldalát tárgyalja, de ahol szükséges, kitér az engine és a frontend felé mutató csatlakozási pontokra is.

2.2 Fejlesztőkörnyezet

2.2.1 Hardverkörnyezet

A fejlesztés során a csapat tagjai az alábbi hardverkörnyezeteket használták:

Fejlesztő	Processzor	Videókártya	RAM	Operációs rendszer
Eördögh Erik	AMD Ryzen 5 7500F	AMD RX 6600 8 GB	16GB	Windows 11 Pro
Pintér Bálint	Intel i7-9700	NVIDIA GeForce RTX 2070 SUPER 8 GB	16GB	Windows 11 Pro
Varga Norbert	Intel i5-13500	NVIDIA GeForce RTX 4060 TI 8 GB	32GB	Windows 11 Home

2.2.2

2.2.3 Szoftverkörnyezet

A fejlesztés során az alábbi szoftvereszközöket használtuk:

Szoftver	Felhasználás célja
Visual Studio Code	Kódszerkesztő (frontend, backend)
Node.js	Szerver futtatókörnyezet
npm	Csomagkezelő
Xampp / Docker	Adat-bázis kezelés
Git	Verziókezelés
Github	Távoli repository
Google Chrome / Microsoft Edge / Opera GX	Böngésző tesztelés, DevTools
Jest	Javascript unit tesztelés
Emscripten SDK	C++-ból Webassembly fordító

2.2.4 A technológiai választás indoka

A backend megvalósításához a Node.js és az Express keretrendszer mellett döntöttünk. Az Express könnyűsúlyú, minimalista keretrendszer, amely elegendő rugalmasságot biztosít egy RESTful API kialakításához anélkül, hogy felesleges komplexitást vinne a projektbe.

Az adatbázis oldalán a MySQL relációs adatbázis-kezelőre esett a választás, mivel a játékméleti adatok (felhasználók, pályák, pontok, tippek) természetüknél fogva táblázatos és relációs szerkezetűek. A MySQL széles körben elterjedt, jól dokumentált és a képzés során is megismert technológia.

A 360 fokos panorámakép-renderelő motor C++ nyelven készült, és WebAssembly-re fordítottuk az Emscripten SDK segítségével. A C++ mellett szólt a teljesítményigényes grafikai számítások hatékony kezelése, a WebGL közvetlen elérése és a memória feletti precíz kontroll. A WebAssembly technológia lehetővé teszi, hogy a natív szintű teljesítményű kódot közvetlenül a böngészőben futtassuk, így a felhasználóknak semmilyen telepítésre nincs szükségük.

2.3 Kialakított adatszerkezetek

2.3.1 Adatbázis-séma áttekintése

A rendszer MySQL adatbázist használ. Az adatbázis UTF-8 Hungarian collation beállítást használ. A táblastruktúra a felhasználói fiókokat, a létrehozott játékpályákat, a pályákon elhelyezett pontokat, a pontok közötti kapcsolatokat, a játékméleti állapotot és a kiegészítő adatokat külön egyedekre bontja.

A rendszer a felhasználói fiókok megszüntetésekor egy 'soft delete' és anonimizációs stratégiát alkalmaz. Fióktörlés esetén a *users* táblában lévő rekord fizikailag nem törlődik, csupán a *deleted_at* mező kap egy időpontot. Ezzel egyidejűleg az alkalmazás logikája végrehajtja a személyes adatok eltávolítását: a *username* és *email* mezőket anonim, generált adatokkal felülírja, a *password* mező tartalmát kiüríti, továbbá megszünteti a profilkép (*ppp*) csatolását is. Ennek az architektúrának a fő előnye, hogy a fizikai sor megtartásával a hivatkozó külső kulcsok (például a létrehozott pályáknál, hozzászólásoknál vagy lejátszott játékmeneteknél) épek maradnak. Így az adatbázis relációs integritása és a globális statisztikák pontossága úgy tartható fenn, hogy közben az eredeti felhasználót azonosító minden információ véglegesen eltűnik a rendszerből.

2.3.3 Játéktérkép és képi erőforrások

A rendszer kettéválasztja a globális pályagyűjtemény és a konkrét bejárható területek fogalmát. A *game_maps* tábla a létrehozott fő játékpályák (például egy teljes kampány) adatait tartalmazza: cím, leírás, készítő, borítókép és létrehozási időpont.

A pályák részterületekre bonthatók: egy *game_maps* egyedhez több térkép (*map*) is tartozhat, egy térképen pedig több pont (*points*) helyezkedhet el. Az *images* tábla a feldolgozott képek adatait (szélesség, magasság, fájlvonal) tartja nyilván. Ez lehetővé teszi, hogy a backend a fájlnév helyett egy egységes azonosító-rétegen keresztül hivatkozzon a képi erőforrásokra.

2.3.4 Pontok és kapcsolatok

A *points* tábla a térképeken elhelyezett panorámaképeket reprezentálja. A pontpozíciók normalizált 2D-s UV koordinátákkal vannak tárolva: a *point_u* és *point_v* értékek [0, 1[intervallumba esnek, ahol a koordináta-rendszer origója (0,0) a panorámakép bal felső sarka. A képpontbeli koordinátákat úgy kapjuk meg, hogy a térkép képének méreteivel beszorozzuk az UV koordinátákat. A *north_direction* attribútum [0, 360[fokban tartalmaz egy eltolási értéket, amely a képen belüli északi irány beállítására szolgál. A koordináták egyedi kombinációja térképenként táblaszintű megszorítással védett, így ugyanazon térképen nem jöhet létre duplikált pont.

A pontok közötti navigációs gráfot a *point_connections* tábla írja le. A navigációs nyilak irányának meghatározása a végpontok helyzetétől függ: ha a két összekötött pont ugyanazon a térképen helyezkedik el, a szögek (*direction_start_to_end* és *direction_end_to_start*) külön megadása szükségtelen (értékük NULL). Ilyenkor az alkalmazás a két pont UV koordinátáiból számított képpont koordinátákkal, geometriai számítással határozza meg az irányokat. Ha

azonban a kapcsolat két eltérő térképet köt össze (pl. épületbe való belépés), a koordinátákból nem számítható valós irány, így ezeket a szögeket a pálya készítőjének kell manuálisan rögzítenie a táblában. A táblaszintű *check_point_order* megszorítás az azonos párokat rendezett formában tárolja, ami egyszerűsíti a gráf éleinek egyediség-kezelését.

2.3.5 Játékmeneti állapot

A *game_sessions* tábla a folyamatban lévő játékfolyamat aktuális állapotát tárolja. A játékmenet logikája körökre (*current_round*) és ciklusokra (*current_cycle*) tagolódik. Egy kör egy konkrét tippelési fázist jelent (egy kép – egy tipp). A ciklus egy ismétlődés-gátló mechanizmus: garantálja, hogy ha egy játékban több kör van, mint ahány egyedi pont a pályán, egy cikluson belül egy kép csak egyszer jelenjen meg. Ha a pontok elfogynak, a ciklus inkrementálódik.

Ebben az egyedben kapott helyet a másodpercben értendő köridő (*time_per_round*).

A *session_guesses* tábla minden leadott tippet eltárol: session, kör, ciklus, tippelt térkép, tippelt koordináták, a kiosztott pontszám és a tipp ideje. Ez teszi lehetővé a részletes játékvégi statisztikák regenerálását.

2.3.6 Kiegészítő adatszerkezetek

A *favorites* tábla a felhasználók kedvenc pályáit tárolja összetett elsődleges kulccsal. A *game_maps_comments* tábla kezeli az értékeléseket és a szöveges hozzászólásokat; az „egy felhasználó egy pálya” szabályt adatbázis-szintű egyediségi megszorítás biztosítja. A leadott értékelések (*rating*) adatminőségét egy további táblaszintű megszorítás (CHECK) garantálja, amely csak az [1; 5] zárt intervallumba eső egész számokat fogadja el érvényes bemenetként. Az adminisztrációs felület visszakövethetőségét a *log* tábla biztosítja. A rendszer megkülönbözteti a felhasználó által indított eseményeket (pl. 'Login', 'Password update') és az adminisztrátori beavatkozásokat (pl. 'User delete (A)', 'Profile picture update (A)'). A *victim_id* oszlop mutatja meg azt a célfelhasználót, akin az adminisztrátor az adott módosítást végrehajtotta. A séma üzembiztonsági szempontból tartalmaz egy MySQL Event Scheduler eseményt is (*expire_abandoned_sessions*). Ez a rutin háttérfolyamatként gondoskodik a legalább két órája inaktív, elárvult sessionök automatikus lezárásáról, garantálva az adatbázis konzisztenciáját és a statisztikák pontosságát.

2.3.7 A C++ WebAssembly motor belső adatszerkezetei

Az alkalmazás működésének kulcsfontosságú részét képezik a C++ alapú renderelő motor futásidőben felépülő, memóriában tárolt adatszerkezetei. Ezek az objektum-orientált struktúrák

felelnek a 3D-s tér, a gömbszegmensek és a 2D-s térképek geometriai reprezentációjáért, valamint a grafikus hardverrel (WebGL/OpenGL) való kommunikációért.

2.3.7.1 Csúcspontok reprezentációja

A 3D-s és 2D-s megjelenítés legkisebb építőköve a *Vertex* struktúra. Ez egyetlen csúcspont adatait fogja össze a memóriában. Minden csúcspont eltárolja a pontos térbeli elhelyezkedését (egy háromdimenziós, lebegőpontos x, y, z koordináta-rendszerben), valamint a textúrázáshoz (a képek felületre feszítéséhez) szükséges kétdimenziós u és v koordinátákat. Ez a tömör adatszerkezet biztosítja, hogy a mérete a memóriában minimális maradjon.

2.3.7.2 Poligonhálók és geometriai adatok

A komplexebb térbeli testek (például a panorámaképeket megjelenítő gömbszegmensek) és a sík felületek kezelését a *Mesh* (poligonháló) osztály végzi. A *Mesh* osztály felelős egy teljes térbeli objektum adatainak összefogásáért és a GPU felé történő továbbításáért:

- **Geometriai adatok:** Dinamikus tömbökben tárolja a testet felépítő csúcspontokat valamint az ezeket háromszögekké összekötő indexeket.
- **Példányosítás (Instancing):** A motor támogatja az úgynevezett "*instanced rendering*" technikát, amely lehetővé teszi, hogy ugyanazt a geometriát egyszerre több helyen is hatékonyan kirajzolja. Ehhez az osztály tárolja a példányok eltolási vektorait.
- **Hardveres pufferek:** A grafikus kártyával való adatcseréhez a *Mesh* osztály kezeli az OpenGL azonosítókat: a Vertex Array Object (**VAO**), Vertex Buffer Object (**VBO**), Element Buffer Object (**EBO**) és az instanciák pufférének (**Instance VBO**) memóriacímeit.
- **Transzformáció és Anyagok:** A *MeshData* struktúrán keresztül tartalmazza az objektum modellmátrixát (*modelMatrix*), amely a térbeli forgatást, eltolást és skálázást írja le. Emellett minden poligonháléhoz tartozik egy anyag (*Material*), amely a megjelenés vizuális tulajdonságait határozza meg.

2.3.7.3 A Színtér felépítése

A *Scene* osztály a teljes renderelendő világ konténereként funkcionál. Célja, hogy logikailag összefogja a képernyőn megjelenő összes elemet:

- **Kamera:** Exkluzív tulajdonlású intelligens mutatóval tartalmazza a nézőpontot meghatározó kamerát, amely a felhasználó forgási és navigációs interakcióit fordítja le a 3D-s térben.
- **Objektumlista:** Egy megosztott mutatókat tartalmazó dinamikus tömbben tartja nyilván az összes aktív poligonhálót. Ez a struktúra teszi lehetővé a motor számára, hogy a renderelési ciklus során itéráljon a kirajzolandó elemeken, illetve futásidőben dinamikusan adjon hozzá vagy távolítson el elemeket a memóriából.

2.4 Rendszerarchitektúra

A Lokea alkalmazás négy fő komponensből áll, amelyek két elkülönített futási környezetben helyezkednek el: a böngészőben és a szerveren.

A böngésző oldal tartalmazza a frontend (HTML/CSS/JS) modult, amely a felhasználói felületért felel, és az Engine modult, amely egy C++ nyelven írt, WebAssembly-re fordított renderelő motor. Ez utóbbi végzi a 360 fokos panorámaképek megjelenítését egy WebGL Canvas elemen. A két modul közvetlen JavaScript API-n keresztül kommunikál egymással.

A szerver oldal a Node.js/Express alapú REST API, amely HTTP kéréseken keresztül fogadja a A backend részletes bemutatása frontend JSON formátumú kéréseit és JSON-nel válaszol. A szerver kezeli a játékállapotot memóriában tárolt munkamenetekkel (session), a képfeltöltéseket a Multer és Sharp könyvtárakkal, a felhasználókezelést és az e-mail küldést SMTP-n keresztül. Az adatbázis-kapcsolatokat MySQL connection pool biztosítja.

Az alkalmazás kétféle adattárolást használ: a strukturált adatok (felhasználók, helyszínek, tippek, pontszámok) MySQL adatbázisban találhatók, míg a 360 fokos képek a szerver fájlrendszerén tárolódnak, és a frontend közvetlenül HTTP-n keresztül kéri le őket.

2.5 A backend részletes bemutatása

2.5.1 Projektstruktúra

A backend moduláris felépítésű. A fő belépési pont a szerver indítása, amely az Express alkalmazást konfigurálja, a middleware-eket regisztrálja és az útvonalakat betölti. A forráskódban törekedtünk a funkcionális egységekre (modulokra) bontásra: sok modul saját controller, service, queries és routes fájlokkal rendelkezik.

2.5.2 REST API végpontok

A backend RESTful architektúrát követ: az erőforrások HTTP metódusokkal (GET, POST, PUT, DELETE) érhetők el, az adatsere JSON formátumban történik. A végpontjaink nevei követik a RESTful API elnevezési irányelveket követi. Az alábbiakban a végpontcsoportok:

2.5.2.1 Auth (/api/auth)

Útvonal	Metódus	Leírás
/api/auth/register	POST	Regisztráció
/api/auth/login	POST	Bejelentkezés
/api/auth/logout	DELETE	Kijelentkezés
/api/auth/status	GET	Bejelentkezési állapot lekérése

2.5.2.2 Felhasználók (/api/users)

Útvonal	Metódus	Leírás
/api/users/language	GET	Nyelvi beállítás lekérése
/api/users/me	GET	Saját profil lekérése
/api/users/me	PUT	Profil frissítése (felhasználónév, email, nyelv, darkmode)
/api/users/me/password	PUT	Jelszó módosítása
/api/users/me	DELETE	Fiók deaktiválása
/api/users/me/profile-picture	GET	Saját profilkép lekérése
/api/users/me/profile-picture	PUT	Profilkép feltöltése
/api/users/me/profile-picture	DELETE	Profilkép törlése
/api/users/:id/profile-picture	GET	Más felhasználó profilképének lekérése
/api/users/profile-picture	GET	Profilkép lekérése fájlnev alapján

2.5.2.3 Lobby (/api/lobby)

Útvonal	Metódus	Leírás
/api/lobby/	GET	Játékpályák listája (szűrés, lapozás)
/api/lobby/session	GET	Aktív játékmenet adatai
/api/lobby/session	POST	Új játékmenet indítása
/api/lobby/session	DELETE	Játékmenet befejezése, végső pontszám visszaadása

2.5.2.4 Játékmenet (/api/game)

Útvonal	Metódus	Leírás
/api/game/session	GET	Jelenlegi játékmenet adatai

/api/game/maps	GET	Elérhető térképek lekérése
/api/game/round	GET	Aktuális kör véletlenszerű pontja
/api/game/round/guess	POST	Tipp beküldése az aktuális körre
/api/game/session	DELETE	Játékmenet befejezése

2.5.2.5 Játékpályák (/api/game-maps)

2.5.2.5.1 Alap CRUD

Útvonal	Metódus	Leírás
/api/game-maps/:gameMapID	GET	Pálya részleteinek lekérése
/api/game-maps/	POST	Új pálya létrehozása
/api/game-maps/:gameMapID	PUT	Pálya metaadatainak frissítése
/api/game-maps/:gameMapID	DELETE	Pálya törlése

2.5.2.5.2 Borítókép

Útvonal	Metódus	Leírás
/api/game-maps/:gameMapID/cover-image	GET	Borítókép lekérése
/api/game-maps/:gameMapID/cover-image	PUT	Borítókép feltöltése
/api/game-maps/:gameMapID/cover-image	DELETE	Borítókép törlése

2.5.2.5.3 Kommentek

Útvonal	Metódus	Leírás
/api/game-maps/:gameMapID/comments	GET	Pálya összes kommentje
/api/game-maps/:gameMapID/my-comment	GET	Saját komment lekérése
/api/game-maps/:gameMapID/my-comment	POST	Komment létrehozása
/api/game-maps/:gameMapID/my-comment	PUT	Komment frissítése
/api/game-maps/:gameMapID/my-comment	DELETE	Komment törlése

2.5.2.5.4 Kedvencek

Útvonal	Metódus	Leírás
/api/game-maps/:gameMapID/favorite	GET	Kedvenc-e a pálya
/api/game-maps/:gameMapID/favorite	POST	Kedvencekhez adás
/api/game-maps/:gameMapID/favorite	DELETE	Kedvencekből eltávolítás

2.5.2.5.5 Képek és útvonalak

Útvonal	Metódus	Leírás
/api/game-maps/points/:pointID/image	GET	Pont panoráma képe
/api/game-maps/maps/:mapID/image	GET	Térkép statikus képe
/api/game-maps/points/:pointID/paths	GET	Pont kapcsolódási irányai

2.5.2.6 Térképkészítő (/api/map-creator)

2.5.2.6.1 Térképek

Útvonal	Metódus	Leírás
/api/map-creator/game-maps/:gameMapID/maps	GET	Gamemap összes térképe

/api/map-creator/game-maps/:gameMapID/maps	POST	Új térkép létrehozása (képfeltöltéssel)
/api/map-creator/maps/:mapID	PUT	Térkép frissítése
/api/map-creator/maps/:mapID	DELETE	Térkép törlése

2.5.2.6.2 Pontok

Útvonal	Metódus	Leírás
/api/map-creator/maps/:mapID/points	GET	Térkép összes pontja
/api/map-creator/maps/:mapID/points	POST	Új pont létrehozása (equirectangular képpel)
/api/map-creator/points/:pointID	PUT	Pont frissítése
/api/map-creator/points/:pointID	DELETE	Pont törlése

2.5.2.6.3 Kapcsolatok

Útvonal	Metódus	Leírás
/api/map-creator/game-maps/:gameMapID/connections	GET	Gamemap összes kapcsolata
/api/map-creator/game-maps/:gameMapID/connections	POST	Új kapcsolat létrehozása
/api/map-creator/connections/:connectionID	PUT	Kapcsolat frissítése
/api/map-creator/connections/:connectionID	DELETE	Kapcsolat törlése

2.5.2.7 Admin panel (/api/admin)

2.5.2.7.1 Dashboard

Útvonal	Metódus	Leírás
/api/admin/dashboard	GET	Áttekintő adatok (felhasználószám, aktívak, logok)
/api/admin/charts/:type	GET	Analitikai diagram generálása

2.5.2.7.2 Felhasználók

Útvonal	Metódus	Leírás
/api/admin/users	GET	Összes felhasználó (lapozva)
/api/admin/users/sorted	GET	Szűrt/rendezett felhasználólista
/api/admin/users/:id	GET	Egy felhasználó adatai
/api/admin/users	POST	Új felhasználó létrehozása
/api/admin/users/exports	POST	Felhasználók exportálása CSV-be
/api/admin/users/self	PUT	Admin saját adatainak frissítése
/api/admin/users/:id	PUT	Felhasználó adatainak frissítése
/api/admin/users/:id/profile-picture	PUT	Felhasználó profilképének feltöltése
/api/admin/users/:id	DELETE	Felhasználó deaktiválása

/api/admin/users/:id/profile-picture	DELETE	Felhasználó profilképének törlése
--------------------------------------	--------	-----------------------------------

2.5.2.7.3 Logok

Útvonal	Metódus	Leírás
/api/admin/logs	GET	Összes aktivitásnapló (lapozva)
/api/admin/logs/sorted	GET	Szűrt naplók
/api/admin/logs	POST	Manuális naplóbejegyzés
/api/admin/logs/exports	POST	Logok exportálása CSV-be

2.5.2.7.4 Beállítások

Útvonal	Metódus	Leírás
/api/admin/settings	GET	Admin dashboard beállításai
/api/admin/settings	PUT	Beállítások frissítése
/api/admin/settings/dark-mode	PUT	Sötét mód váltása
/api/admin/settings/language	PUT	Nyelv váltása

2.5.3 Hitelesítés és munkamenet kezelés

A felhasználói hitelesítés session-alapú megoldással működik. Bejelentkezéskor a rendszer bcrypt segítségével összehasonlítja a megadott jelszót a tárolt hash-sel, majd sikeres egyezés esetén létrehoz egy munkamenetet, amelyben a felhasználó azonosítóját és jogosultsági szintjét tárolja. Az adminisztrátori és LORD szintű végpontokat middleware-ek védik, amelyek a session role mezője alapján döntenek a hozzáférésekről.

2.5.4 Képfeldolgozás

A panorámaképek és borítóképek feltöltéséért a **Multer** middleware felel, amely az Express kérésekből kinyeri a multipart/form-data formátumú fájlokat. A feltöltött képeket a **Sharp** könyvtár dolgozza fel: az alkalmazás átméretezi, tömöríti és optimalizálja a képeket a megjelenítés előtt. A feldolgozott fájlok a szerver fájlrendszerén kerülnek eltárolásra, metaadataik (szélesség, magasság, fájlvonal) pedig az images táblába mentődnek.

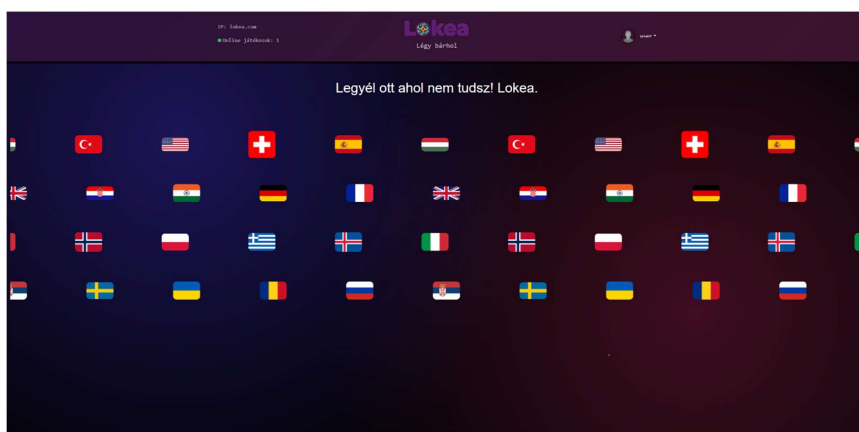
2.6 A frontend bemutatása

A kliensoldali alkalmazás HTML5, CSS3 és vanilla JavaScript technológiákra épül. Az oldalak közötti navigáció szerver által kiszolgált HTML oldalakon alapul, az interaktív funkciók (játékmenet, térkép kezelés, tipp leadás) pedig aszinkron fetch hívásokkal kommunikálnak a REST API-val.

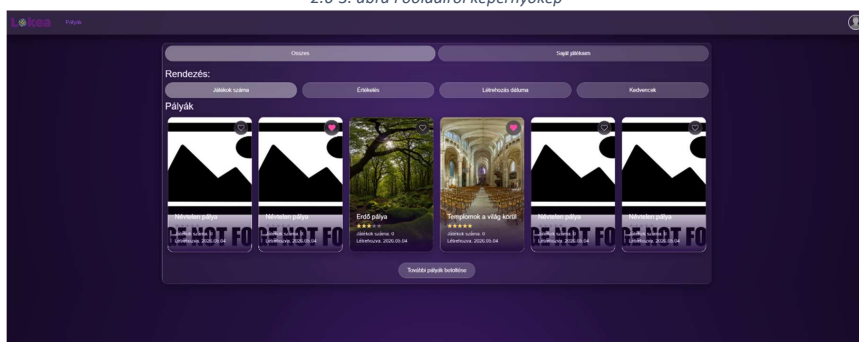
A felhasználói felület főbb nézetei:

- **Főoldal:** Felhasználói beállítások, a megfelelő jogosultsággal rendelkezőknek az adminoldal elérése, népszerű pályák kiemelése.

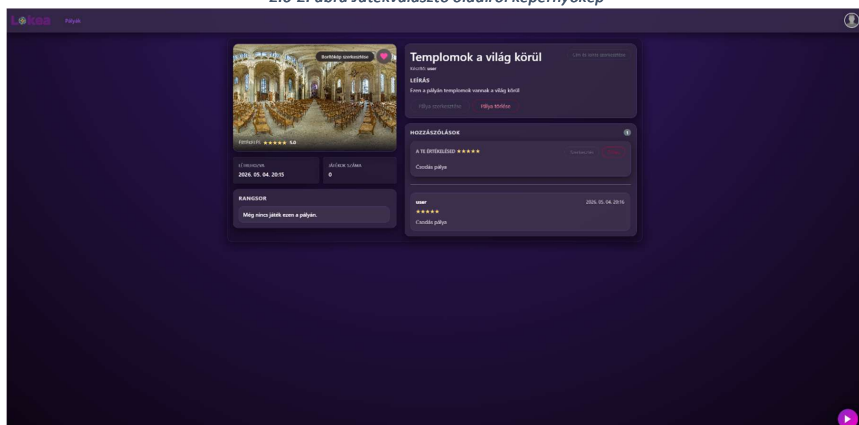
- **Játékválasztó:** Játékok közül választás, rendszerezésük különböző szempontok alapján, saját pályák létrehozása
- **Pályaoldal:** A kiválasztott játék előnézete, toplistával, kommentelési lehetőséggel, ha a felhasználó csinálta a pályát, a változtatás lehetősége is elérhetővé válik
- **Játékoldal:** A 360 fokos panorámakép megjelenítése az Engine modul segítségével, a térkép és a tipp leadó felület. A játék végén itt is van toplista, lehetséges a kommentelés.
- **Pályaszerkesztő:** Térképek feltöltése, pontok elhelyezése, kapcsolatok létrehozása.
- **Profil modal:** Felhasználói adatok, oldalbeállítások(sötét-világos mód, oldal nyelve)
- **Adminisztrációs felület:** Felhasználókezelés, statisztikai vezérlőpult, térkép, panorama kép tesztoldalak elérése.
 - **Panoráma-kép tesztoldal:** Az equirectangular engine funkcióinak tesztelése
 - **Térkép tesztoldal:** A mapviewer engine funkcióinak tesztelése



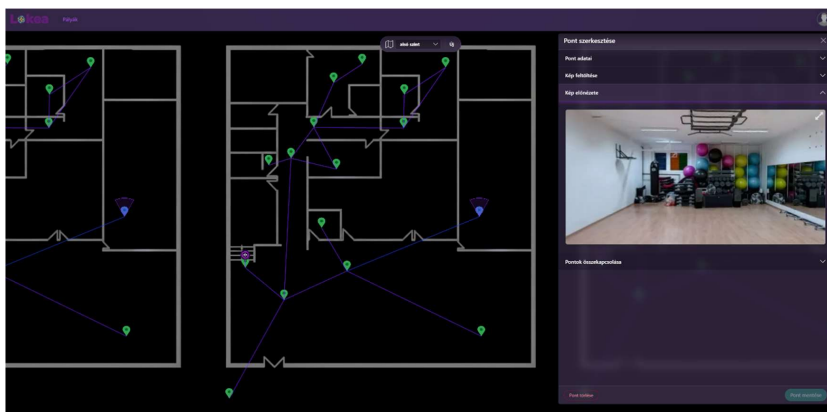
2.6-3. ábra Főoldalról képernyőkép



2.6-2. ábra Játékválasztó oldalról képernyőkép



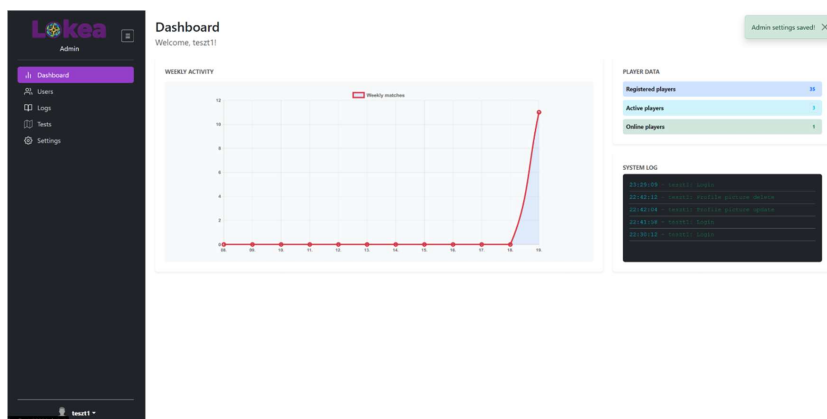
2.6-1. ábra Pályaaloldalról képernyőkép



2.6-6. ábra Pályakészítő oldalról képernyőkép



2.6-5. ábra Játéktoldalról képernyőkép



2.6-4. ábra Admin felületről képernyőkép

2.7 WebAssembly Engine

A Lokea egyik legegységibb komponense a saját fejlesztésű 360 fokos panorámakép-renderelő motor, amely C++ nyelven készült és WebAssembly-re fordítva fut a böngészőben.

2.7.1 Az Engine felépítése

A motor az **engine/** könyvtárban található. A fordítás az Emscripten SDK-val és CMake-kel történik. Az engine közvetlenül a böngésző WebGL API-ját használja a 3D-s rendereléshez: egy gömbfelületre vetíti az equirektanguláris panorámaképet, majd a felhasználó egérrel vagy érintéssel forgathatja a nézőpontot. A rendereléshez GLSL shadereket (vertex és fragment shader) használ.

2.7.2 A motor feladatai

- **Panorámakép megjelenítése:** Az equirektanguláris kép gömbvetítése és valós idejű forgatása.
- **Térkép renderelése:** A 2D-s térképkép megjelenítése és interaktív kezelése (görgetés, nagyítás).
- **Navigációs nyilak megjelenítése:** A pontok közötti kapcsolatok vizuális jelölése a panorámanézetben.
- **Teljesítmény:** A C++/WASM kombináció közel natív sebességet biztosít, ami különösen nagy felbontású képek esetén fontos.

2.8 A program tipikus algoritmusai

2.8.1 Panorámaképek gömbszegmens-alapú renderelése

A C++/WebAssembly alapú motor egyik legkomplexebb algoritmus a 360 fokos panorámaképek (equirectangular textúrák) megjelenítéséért felelős gömbfelület generálása. A böngészős WebGL környezet hardveres korlátjai miatt a motor nem egyetlen egybefüggő gömböt renderel, hanem a felületet kisebb, önálló textúratartományokkal rendelkező gömbszegmensekre bontja.

2.8.1.1 Dinamikus csempézés és textúrakezelés

Amikor a kliens betölt egy panorámaképet, az algoritmus első lépésként lekérdezi a videokártya (GPU) által támogatott maximális textúraméretet (`GL_MAX_TEXTURE_SIZE`). Amennyiben a forráskép felbontása meghaladja ezt a hardveres korlátot, a rendszer automatikusan felbontja a textúrát egy 2x2-es vagy 4x4-es rácshálónak megfelelő csempékre.

Ennek a döntésnek megfelelően a geometriai generátor is pontosan annyi különálló, de térben tökéletesen illeszkedő gömbszegmenst állít elő, ahány csempére a képet bontani kellett. Ez a megközelítés garantálja, hogy az alkalmazás a régebbi vagy alacsonyabb memóriával rendelkező eszközökön is képes legyen a nagy felbontású terek megjelenítésére.

2.8.1.2 Matematikai modell és koordináta-transzformáció

Egy adott szegmens előállításánál az algoritmus a gömb felületét egy vertikális (szélességi) és egy horizontális (hosszúsági) rácshálóra osztja fel. A felület pontjainak kiszámítása egy kétdimenziós iteráció során történik:

- **Vertikális iteráció (Szélességi fokok):** Az algoritmus a gömbön való függőleges előrehaladást egy V (vertikális) textúrákoordinátára képezi le. Ebből a normalizált értékből határozza meg a polárszöget, amely a $[0, \pi]$ intervallumba esik (ahol a 0 az északi pólust, a $\pi/2$ az egyenlítőt, a π pedig a déli pólust jelképezi).
- **Horizontális iteráció (Hosszúsági fokok):** A vízszintes lépések során az algoritmus meghatározza az U (horizontális) textúrákoordinátát. Ezt felhasználva számítja ki az azimutszöget a $[0, 2\pi]$ intervallumon, amely az Y -tengely körüli teljes 360 fokos fordulatot írja le.

Ezekből a gömbi koordinátákból (polárszög, azimutszög és az előre definiált gömbsugár) a rendszer trigonometriai transzformációk segítségével kiszámítja a térbeli, derékszögű (Descartes-koordináta-rendszerbeli) X , Y és Z koordinátákat.

2.8.1.3 A poligonháló felépítése

A kiszámított térbeli pontokból az algoritmus létrehozza a csúcspontok (vertexek) tömbjét, ahol minden egyes térbeli koordináta-hoz szorosan hozzárendeli a korábban kiszámított U és V textúrákoordinátákat.

A renderelhető 3D-s háló véglegesítéséhez a motornak meg kell határoznia a pontok közötti kapcsolatokat is. Ezt az indexpuffer felépítésével éri el: a rácsháló szomszédos csúcspontjait virtuális négyszögekké fogja össze, majd minden egyes négyszöget két szomszédos, GPU által értelmezhető háromszögre bont.

Ez az optimalizált, szegmensekre bontott eljárás teszi lehetővé, hogy ugyanaz a geometriai logika újrahasznosítható legyen függetlenül attól, hogy a teljes gömböt, vagy csak annak egyetlen felbontott szeletét kell a memóriába tölteni.

2.8.2 Az oldal kétnyelvűsítése (Lokalizáció)

A Lokea alkalmazás globális elérhetőségéből és a felhasználói élmény maximalizálásából adódóan a kliensoldali felület többnyelvű (jelenleg kétnyelvű) támogatással rendelkezik. A lokalizáció (i18n) technikai megvalósítását az adatbázis-szintű nyelvi preferenciák és a frontend rétegben integrált i18next könyvtár együttesen biztosítják.

```
1 {
2   "apiMain": {
3     "signup": {
4       "validation_username_no_email": "A felhasználónév nem lehet e-mail cím!",
5       "validation_username_invalid_chars": "A felhasználónév csak betűket, számokat, _ és - karaktereket tartalmazhat.",
6       "validation_username_length": "A felhasználónév hossza 1 és 20 karakter között kell legyen.",
7       "validation_email_format": "Hibás e-mail formátum.",
8       "validation_email_length": "Az e-mail cím hossza 5 és 254 karakter között kell legyen.",
9       "validation_password_length": "A jelszó hossza 8 és 60 karakter között kell legyen.",
10      "validation_password_digit": "A jelszónak tartalmaznia kell legalább egy számot!",
11      "validation_password_uppercase": "A jelszónak tartalmaznia kell legalább egy nagybetűt!",
12      "success": "Sikerrel regisztráció!",
13      "user_exists": "A felhasználónév vagy e-mail cím már foglalt!",
14      "error": "Hiba történt a regisztráció során!"
15    },
16    "login": {
17      "validation_username_length": "A felhasználónév/e-mail hossza nem megfelelő!",
18      "validation_password_length": "A jelszó hossza nem megfelelő!",
19      "invalid_credentials": "Hibás felhasználónév/e-mail vagy jelszó!",
20      "success": "Sikerrel bejelentkezés!",
21      "error": "Hiba a bejelentkezés során!"
22    },
23    "language_fetch_error": "Hiba a nyelv lekérdezésekor"
24  },
25  "apiSettings": {
26    "getUserData": {
27      "error": "Hiba a felhasználói adatok lekérdezése során!"
28    }
29  }
30 }
```

```
1 {
2   "apiMain": {
3     "signup": {
4       "validation_username_no_email": "Username cannot be an e-mail address!",
5       "validation_username_invalid_chars": "Username can only contain letters, numbers, _ and - characters.",
6       "validation_username_length": "Username must be between 1 and 20 characters long!",
7       "validation_email_format": "Invalid e-mail format.",
8       "validation_email_length": "E-mail must be between 5 and 254 characters long!",
9       "validation_password_length": "Password must be between 8 and 60 characters long!",
10      "validation_password_digit": "Password must contain at least one number!",
11      "validation_password_uppercase": "Password must contain at least one uppercase letter!",
12      "success": "Registration successful!",
13      "user_exists": "Username or e-mail is already taken!",
14      "error": "Error during registration!"
15    },
16    "login": {
17      "validation_username_length": "Username/e-mail length is invalid!",
18      "validation_password_length": "Password length is invalid!",
19      "invalid_credentials": "Invalid username/e-mail or password!",
20      "success": "Login successful!",
21      "error": "Error during login!"
22    },
23    "language_fetch_error": "Error fetching language"
24  },
25  "apiSettings": {
26    "getUserData": {
27      "error": "Error fetching user data!"
28    }
29  }
30 }
```

2.8-1. ábra A szótárfájlok (JSON) szerkezete

2.8.2.1 Adatbázis-oldali támogatás

A perzisztens nyelvi beállítások kezelése rendszerszinten történik. A rendszer a felhasználó által preferált nyelvet a *users* táblában, egy dedikált *language* attribútumban tárolja. Ez az érték a regisztráció során kap egy alapértelmezést, majd a személyes beállításoknál (a sötét módhoz, azaz a darkmode-hoz hasonlóan) bármikor módosítható. A többnyelvű karakterkészletek (például a magyar ékezetes betűk) adatintegritását és megfelelő reprezentációját az adatbázis-szintű UTF-8 Hungarian collation beállítás garantálja.

2.8.2.2 Kliensoldali implementáció (i18next)

A böngészőben futó kliens felületének dinamikus nyelvi fordításait az i18next keretrendszer végzi. Az alkalmazás architektúrája szétválasztja a statikus szövegeket és a forráskódot: a felületi elemek (menüpontok, hibaüzenetek, gombok feliratai) közvetlen szövegmegadás helyett egyedi azonosítókra (fordítási kulcsokra) hivatkoznak.

Amikor a felhasználó sikeresen hitelesíti magát, a backend a válaszüzenetben átadja a *users* táblából kiolvasott *language* mező értékét. Az i18next inicializálása ezen paraméter alapján történik meg. A keretrendszer aszinkron módon betölti a memóriába a megfelelő nyelvi erőforrásokat (például JSON formátumú szótárfájlokat), majd a frontend motor renderelési fázisában a kulcsokat a megfelelő lokalizált értékekre cseréli.

2.8.2.3 Hatásköre és adminisztrációs integráció

A kényelvűsítési logika a rendszer teljes terjedelmére kiterjed. A kliensoldali útválasztás és a játékmeneti instrukciók mellett az adminisztrációs felület megjelenítése is ezen a rendszeren keresztül történik.

2.8.3 Pontszámítás algoritmusa

A játékos minden körben egy tippet küld be: a térkép normalizált koordinátáit ($u, v \in [0, 1]$) és a térképezonosítót (`map_id`). A `guess.service.js` fájlban implementált logika négy lépésben értékeli a tippet.

Érvényességi ellenőrzések. Mielőtt bármilyen számítás elvégzésre kerülne, a rendszer két feltételt vizsgál. Először ellenőrzi, hogy a kör ideje lejárt-e: ha a `roundStartedAt` időbélyeg és a beállított `roundTime` alapján számított körhatáridő már elmúlt, a tipp automatikusan 0 pontot kap. Másodszor megvizsgálja, hogy a tipp ugyanarra a térképre érkezett-e, amelyen a helyes pont található, illetve hogy a koordináták a $[0, 1]$ tartományon belül vannak-e. Amennyiben bármelyik feltétel sérül, a kör szintén 0 ponttal zárul.

Pixeltávolság kiszámítása. A rendszer lekéri a helyes térkép pixelméreteit (`width`, `height`), és mindkét tengelyen kiszámítja a tipp és a helyes pont eltérését:

$$\begin{aligned}\Delta x &= (u_{\text{tipp}} - u_{\text{helyes}}) \cdot \text{width} \\ \Delta y &= (v_{\text{tipp}} - v_{\text{helyes}}) \cdot \text{height} \\ d &= \sqrt{\Delta x^2 + \Delta y^2}\end{aligned}$$

Pontszám képlete. A pontszámot a távolság és a térkép átlójának hányadosán alapuló Gauss-görbe határozza meg. Az átló (D) normalizálja a távolságot, hogy különböző méretű térképeken azonos nehézségű legyen a játék:

$$\begin{aligned}D &= \sqrt{\text{width}^2 + \text{height}^2} \\ d^{\wedge} &= \max(0, d/D - 0.01) \\ \text{score} &= \text{round}(5000 \cdot e^{(-35 \cdot d^{\wedge}^2)})\end{aligned}$$

A 0.01-es `BULLSEYE_FRACTION` egy kis türelmi zónát biztosít: ha a tipp az átló 1%-án belül esik a helyes ponttól, a játékos teljes (5000) pontszámot kap. A -35 -ös `SHARPNESS` konstans a görbe meredekségét szabályozza: az átló mintegy 20%-ánál a kapott pont már csak ≈ 400 , a pontfelezés pedig $\approx 15\%$ -nál történik.

2.8.4 Cikluskezelés: ismétlődésmentes pontkiosztás

A játékmenet során a rendszer garantálja, hogy egy cikluson belül minden panorámapont legfeljebb egyszer jelenjen meg. A véletlenszerű pontkiosztó algoritmus lekéri a pályán elérhető

pontokat, majd kiszűri azokat, amelyek az aktuális ciklusban már szerepeltek (a `session_guesses` tábla alapján). Ha a szűrt lista kiürül (azaz minden pont felhasználásra került), a ciklus inkrementálódik, és a teljes pontkészlet újra elérhetővé válik. Ez a mechanizmus lehetővé teszi, hogy a játékos a játékhoz tartozó pontok számánál több kört játszhasson anélkül, hogy ki kelljen lépnie a pályáról.

2.8.5 Navigációs irány számítása

Két pont közötti navigációs nyíl irányát a rendszer az UV koordinátaikból számítja ki, amennyiben mindkét pont ugyanazon a térképen helyezkedik el. A számítás az `atan2` függvénnyel történik a pixelkoordináták különbségéből, az eredmény fokban, a `north_direction` eltolás figyelembevételével kerül megjelenítésre. Ha a két pont eltérő térképeken van (pl. épületbe való belépés), az automatikus számítás nem lehetséges, ilyenkor a pályakészítő manuálisan adja meg az irányokat.

2.8.6 Képfeldolgozás menete

A feltöltött panorámaképek egy feldolgozási láncon haladnak keresztül. A Multer middleware fogadja a multipart/form-data kérést és átmenetileg elmenti a fájlt. Ezt követően a Sharp könyvtár elvégzi az átméretezést (a maximális felbontás korlátozása a szerver erőforrásainak védelme érdekében), majd optimalizált formátumban (JPEG/WebP) menti el a végső fájlt. Az `images` táblába a feldolgozott kép metaadatai (szélesség, magasság, elérési útvonal) kerülnek.

2.8.7 Törölt felhasználók kezelése és anonimizáció

A felhasználói fiókok törlése során a rendszer nem fizikai törlést, hanem anonimizációt hajt végre. Az algoritmus a következő lépéseket hajtja végre tranzakcióban: a `deleted_at` mező beállítása az aktuális időpontra, a `username` és `email` mezők felülírása generált anonim értékekkel, a jelszó hash kiürítése, valamint a profilkép kapcsolatának megszüntetése. Ezzel az eljárással a hivatkozó külső kulcsok (játékmenetek, kommentek, pályák) épek maradnak, miközben a személyes adatok visszafordíthatatlanul eltűnnek.

2.9 Tesztdokumentáció

2.9.1 Egységtesztelés (Jest)

A backend logikáját a **Jest** tesztelő keretrendszerrel teszteltük. A tesztek futtatása az **npm test** paranccsal lehetséges. A tesztek az egyes service modulok helyességét vizsgálják, különös

tekintettel a pontszámítási logikára, az érvényességi ellenőrzésekre. A célkitűzésünk az volt, hogy a backend végpontok sorainak, függvényeinek, ágainak 80%-a le legyen tesztelve, ezt sikerült is elérni.

[NV1] megjegyzést írt: Kép a tesztekéről

```

PASS test/admin/apiAdmin.test.js
PASS test/admin/apiLogs.test.js
PASS test/main/apiMain.test.js
PASS test/admin/apiSettings.test.js
PASS test/main/apiSettings.test.js
PASS test/game/gameLobby.test.js
PASS test/game/game.test.js
PASS test/admin/apiUsers.test.js
PASS test/gamemaps/paths.test.js
PASS test/gamemaps/favorite.test.js
PASS test/gamemaps/images.test.js
PASS test/gamemaps/gamemap.test.js
PASS test/gamemaps/cover-image.test.js
PASS test/mapcreator/connections.test.js
PASS test/mapcreator/maps.test.js
PASS test/gamemaps/comments.test.js
PASS test/mapcreator/points.test.js

```

File	% Stats	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.13	84.89	83.33	94.33	
admin	87.73	78.23	69.81	88.34	
apiAdmin.js	87.5	66.66	85.71	87.01	39,55,78-85,116
apiLogs.js	88	84.00	78.57	87.32	26-32,41,117,128-134,139
apiSettings.js	100	100	100	100	
apiUsers.js	84.05	75.32	60.31	85.64	...4-50,102-103,107,153-154,160-164,195,201,209,225,241,247,258,262,279,290-293,296,365,370
index.js	100	100	100	100	
gameflow	98.43	92.85	100	98.43	
game.js	100	100	100	100	
gameLobby.js	94.44	50	100	94.44	17
gameLobby.middleware.js	100	100	100	100	
gameflow/guess	97.53	93.33	100	97.5	
guess.controller.js	100	100	100	100	
guess.routes.js	100	100	100	100	
guess.service.js	96.77	92.85	100	96.72	27,92
gameflow/maps	95	50	100	94.73	
maps.controller.js	90	50	100	90	11
maps.routes.js	100	100	100	100	
maps.service.js	100	100	100	100	
gameflow/random-point	100	100	100	100	
random-point.controller.js	100	100	100	100	
random-point.routes.js	100	100	100	100	
random-point.service.js	100	100	100	100	
gameflow/sessions	95.45	90	100	95.45	
sessions.controller.js	95.45	83.33	100	95.45	18

2.9-1 ábra Teszteredmények 1

sessions.controller.js	95.45	83.33	100	95.45	18
sessions.routes.js	100	100	100	100	
sessions.service.js	94.28	91.66	100	94.28	37,40
gameflow/shared	100	100	100	100	
gameflow.utils.js	100	100	100	100	
gamemaps	92.85	75	100	92.85	
gamemaps.routes.js	92.85	75	100	92.85	52-55
gamemaps/comments	99.16	96	100	99.16	
comments.controller.js	100	100	100	100	
comments.routes.js	100	100	100	100	
comments.schemas.js	100	100	100	100	
comments.service.js	98.36	96	100	98.36	51
gamemaps/cover-image	100	92.85	100	100	
cover-image.controller.js	100	100	100	100	
cover-image.routes.js	100	100	100	100	
cover-image.schemas.js	100	100	100	100	
cover-image.service.js	100	92.1	100	100	108-117,156
gamemaps/favorite	98.64	94.11	100	98.64	
favorite.controller.js	100	100	100	100	
favorite.routes.js	100	100	100	100	
favorite.schemas.js	100	100	100	100	
favorite.service.js	97.61	94.11	100	97.61	33
gamemaps/gamemap	100	95.45	100	100	
gamemap.controller.js	100	100	100	100	
gamemap.routes.js	100	100	100	100	
gamemap.schemas.js	100	100	100	100	
gamemap.service.js	100	95.45	100	100	78
gamemaps/images	100	100	100	100	
images.controller.js	100	100	100	100	
images.routes.js	100	100	100	100	
images.schemas.js	100	100	100	100	
images.service.js	100	100	100	100	
gamemaps/paths	100	100	100	100	
paths.controller.js	100	100	100	100	
paths.routes.js	100	100	100	100	
paths.schemas.js	100	100	100	100	
paths.service.js	100	100	100	100	
gamemaps/shared/schemas	100	100	100	100	
gamemaps.schemas.js	100	100	100	100	
main	78.75	58.75	79.16	78.99	
apiMain.js	86.58	60.23	87.5	86.88	16,21,82-86,100,107,126,133,140-141
apiSettings.js	78.14	53.7	75	78.52	27,29,42,48-51,72-75,97,107,119,130-133,136,178-180,213-218,236-241,249-260
index.js	0	100	100	0	1-10
mapcreator	90.62	80	100	90.62	
mapcreator.js	90.62	80	100	90.62	23-26

2.9-2- ábra Teszteredmények 2

mapcreator.js	90.62	80	100	90.62	23-26
mapcreator/connections	100	100	100	100	
connections.controller.js	100	100	100	100	
connections.routes.js	100	100	100	100	
connections.schemas.js	100	100	100	100	
connections.service.js	100	100	100	100	
mapcreator/maps	100	92.3	100	100	
maps.controller.js	100	100	100	100	
maps.routes.js	100	100	100	100	
maps.schemas.js	100	100	100	100	
maps.service.js	100	92.3	100	100	100,163
mapcreator/points	100	91.07	100	100	
points.controller.js	100	100	100	100	
points.routes.js	100	100	100	100	
points.schemas.js	100	100	100	100	
points.service.js	100	91.07	100	100	96,105-113,196,262
mapcreator/shared/schemas	100	100	100	100	
mapcreator.schemas.js	100	100	100	100	

Test Suites: 23 passed, 23 total
Tests: 1295 passed, 1295 total
Snapshots: 0 total
Time: 5.803 s
Ran all test suites.
%PS C:\Vokea\backend>

2.9-3-ábra Teszteredmények 3

2.9.2 Böngészőtesztelés

Az alkalmazást az alábbi böngészőkben teszteltük

Böngésző	Eredmény
Google Chrome	Minden funkció hibátlanul működik
Opera GX	Minden funkció hibátlanul működik
Microsoft Edge	Minden funkció hibátlanul működik

2.9.3 Validáció és hibakezelés tesztelése

A rendszer viselkedését nem optimális használat esetén is teszteltük. Az alábbi táblázat foglalja magában a főbb teszteseteket.

Teszteset	Bemenet	Elvárt viselkedés	Eredmény
Hibás bejelentkezés	Rossz jelszó	Hibaüzenet, belépés megtagadva	Megfelelő
Duplikált regisztráció	Létező e-mail	Hibaüzenet, regisztráció megtagadva	Megfelelő
Lejárt koridő	Tipp az időlimit után	0 pont kiosztása	Megfelelő
Rossz térkép	Tipp másik térképen	0 pont kiosztása	Megfelelő
Érvénytelen koordináta	$u > 1$ vagy $v < 0$	0 pont kiosztása	Megfelelő
Üres pályán játék	Pálya 0 ponttal	Hibaüzenet, játék nem indul	Megfelelő
Nagy fájl feltöltés	50 MB kép	Elutasítás méretkorlát miatt	Megfelelő
SQL injection kísérlet	Speciális karakterek	Paraméteres lekérdezés véd	Megfelelő

2.9.4 Tesztelői hozzáférés

A program teszteléséhez az alábbi bejelentkezési adatokkal lehet belépni saját regisztráció nélkül:

Felhasználónév: Jelszó:

2.10 Továbbfejlesztési lehetőségek

Az alkalmazás jelenlegi formájában egy működőképes, komplex vizsgaprojekt, amely számos irányban bővíthető. Az alábbiakban néhány lehetséges fejlesztési irányt vázolunk:

- **Natív mobilalkalmazás:** A jelenlegi reszponzív webes kliens mellett egy dedikált Android és/vagy iOS alkalmazás fejlesztése, amely kihasználja az eszköz giroszkópját a 360 fokos panorámakép forgatásához.
- **Többjátékos mód:** Valós idejű, egymás elleni játékmód megvalósítása WebSocket (pl. Socket.io) alapon, ahol a játékosok azonos időben, azonos pályán versenyeznek.
- **Pálya-moderáció és jelentés:** Közösségi moderációs rendszer kialakítása, amelyben a felhasználók jelenthetik a nem megfelelő tartalmú pályákat, és az adminisztrátorok hatékonyan áttekinthetik ezeket.

- **Bajnokságok rendezése:** A különböző felhasználók által létrehozott pályákon többjátékos módokban bajnokságokat is lehetne szervezni, a játék hírnevének növelése érdekében.
- **Statisztikai bővítés:** Részletesebb játékos statisztikák (átlagos pontszám, legjobb/legrosszabb kör, javulási trend), hőtérkép a tippek eloszlásáról pályánként.

3 Felhasználói dokumentáció

3.1 Az alkalmazás célja és főbb funkciói

A Lokea egy böngészőben futó, lokalizációs készségfejlesztő webes alkalmazás, emellett szórakozás céljából is tökéletes választás. A program célja, hogy játékos formában fejlessze a felhasználók téri tájékozódási képességét és térképolvasási készségét.

A játék lényege: a felhasználó egy 360 fokos panorámaképet lát, és ennek alapján kell megjelölnie a feltételezett tartózkodási helyét egy interaktív térképen. Minél pontosabb a tipp, annál több **pontot** kap a játékos (legfeljebb 5000 pontot körönként). A megszerzett pontszámok egy globális ranglistán is megjelennek.

A program főbb funkciói:

- **Játék indítása:** Bármely elérhető pályán, testreszabható beállításokkal (körök száma, köridő).
- **Saját pálya készítése:** Regisztrált felhasználók feltölthetik saját panorámaképeiket, meghatározhatják a pontok helyét a térképen és összeköthetik azokat.
- **Ranglista:** Globális és pályaszintű ranglisták megtekintése.
- **Értékelés és hozzászólás:** A pályák értékelése csillagokkal és szöveges véleménnyel.
- **Profil:** Profil kép beállítása, nyelv beállítása és kinézet téma beállítása.
- **Adminisztráció:** Adminisztrátori jogosultsággal felhasználók tudnak felhasználókat kezelni, statisztikákat tudnak megtekinteni, a naplót megtudják tekinteni és a teszt oldalakon tudják tesztelni a nézegetőket.

3.2 Szükséges hardver és szoftvereszközök

Az alkalmazás használatához az alábbi minimális feltételek szükségesek:

Hardver: Bármely asztali számítógép, laptop, tablet vagy okostelefon, amely rendelkezik WebGL-képes grafikus meghajtóval és internetkapcsolattal. A 360 fokos panorámaképek megjelenítéséhez ajánlott legalább 4 GB RAM és egy közepes teljesítményű grafikus kártya vagy integrált GPU.

Szoftver: Egy modern, WebAssembly-kompatibilis böngésző szükséges. Támogatott böngészők:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge

- Safari

Semmilyen telepítés, plugin vagy kiegészítő nem szükséges a használatához.

3.3 Telepítés és indítás

Felhasználóknak: Az alkalmazás használatához nincs szükség telepítésre. Nyissa meg a böngészőjét, és navigáljon az alábbi címre:

127.0.0.1:3000

Szerver telepítése (fejlesztőknek): Amennyiben az alkalmazást saját gépen szeretné futtatni, az alábbi lépéseket kell követni:

1. Győződjön meg róla, hogy a **Node.js v18** vagy újabb verzió telepítve van a gépére.
2. Klónozza a GitHub repositoryt és telepítse a függőségeket:

```
git clone https://github.com/pinterbalint06/lokea.git
cd lokea/backend
npm install
```

3. Hozzon létre egy **.env** fájlt a backend/ mappában az alábbi környezeti változókkal:

```
SERVER_IP=localhost
PORT=3000
DB_PASS=az_adatbazis_jelszava
EMAIL_USER=email@example.com
EMAIL_PASS=email_jelszava
```

4. Importálja a MySQL adatbázist a database/ mappában található dump fájlból.
5. Indítsa el a szerveret:

```
npm start
```

Az alkalmazás ezután elérhető a megadott IP-címen és porton.

3.3.1 WebAssembly buildelése

Ez a szakasz a C++ alapú WebAssembly motor forráskódjának fordítási (buildelési) folyamatát mutatja be.

3.3.1.1 Követelmények

A sikeres fordításhoz a következő környezeti elemek telepítése szükséges:

1. **CMake:** minimum 3.10-es verzió
2. **Emscripten SKD:** a C++ kód WebAssembly-re (WASM) történő fordításához.

3.3.1.2 A buildelés lépései

1) Build könyvtár létrehozása:

Hozzon létre egy build nevű könyvtárat a motor forráskódjának gyökerében, majd lépjen be a mappába.

```
mkdir build
cd build/
```

2) A build konfigurálása:

Futtassa az emcmake cmake parancsot a build könyvtárból a projekt konfigurálásához.

```
emcmake cmake ..
```

Amennyiben azt szeretné, hogy a folyamat a lefordított build fájlokat automatikusan átmásolja a frontend/javascript mappába, használja a következő kiegészítő kapcsolót:

```
emcmake cmake -DCOPYFILES=ON ..
```

3) Buildelés futtatása: A sikeres konfigurálást követően futtassa a build parancsot.

- o **Adott verzió buildelése:** A Release build a maximális teljesítményre van optimalizálva. A <verzióneve> helyére az alább felsorolt célok egyike helyettesítendő:

```
cmake --build . --target <verzióneve>
```

- o **Mindegyik verzió buildelése:** Az összes elérhető cél egyidejű lefordításához:

```
cmake --build .
```

4) Elérhető verziók

- o A rendszer különböző modulokhoz és tesztelési fázisokhoz eltérő verziókat biztosít:
 - **equirectangular:** Az equirectangular oldalon használt fájlok, amelyek az ekvirectanguláris képek megjelenítésének teszteléséhez készített optimalizált verziót jelentik.
 - **equirectangular_debug:** Az ekvirectanguláris képek megjelenítésének debuggolható (hibakeresésre alkalmas) verziója.
 - **mapViewer:** A test-map oldalon használt, a 2D-s térképek megjelenítésének tesztelésére szánt optimalizált verzió.

- **mapViewer_debug:** A 2D-s térképek megjelenítésének debugolható verziója.

3.4 A program részletes bemutatása

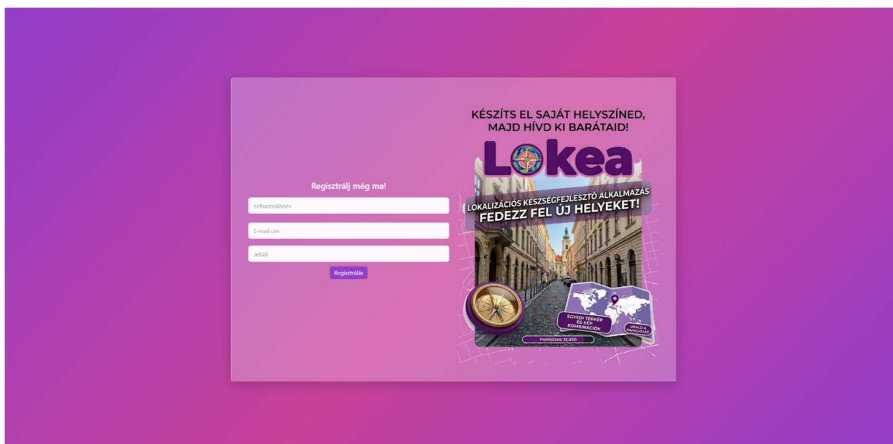
3.4.1 Regisztráció és bejelentkezés

Az alkalmazás regisztráció nélkül korlátozott, csak a főoldalt lehet böngészni. A játék indításához, saját pálya készítéséhez, játékok megnézéséhez, értékeléshez és hozzászóláshoz regisztráció szükséges.

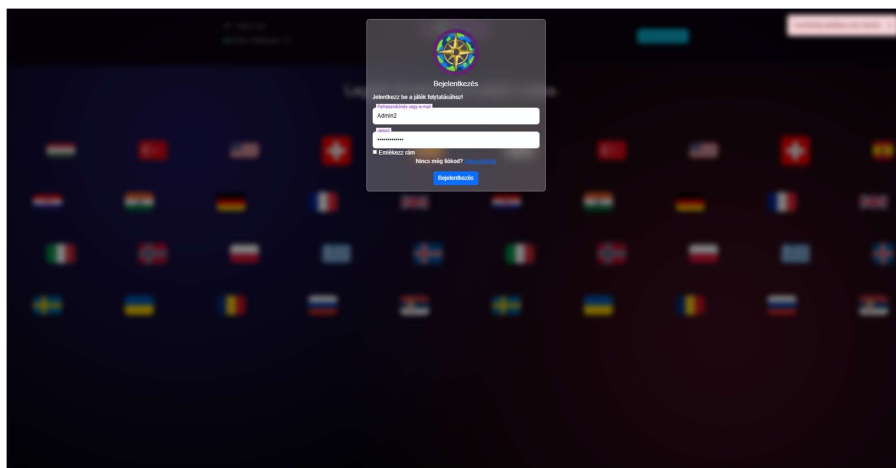
A regisztráció során az alábbi adatokat kell megadni:

- **Felhasználónév:** Egyedi, legalább 3 karakter hosszú, maximum 20.
- **E-mail cím:** Érvényes formátumú, egyedi.
- **Jelszó:** Legalább 8 karakter, tartalmazzon kis- és nagybetűt, valamint számot.

A bejelentkezés a felhasználónév (vagy e-mail) és a jelszó megadásával történik. Hibás adatok esetén a rendszer hibaüzenetet jelenít meg.



3.4-1. ábra Regisztrációs oldalról képernyőkép

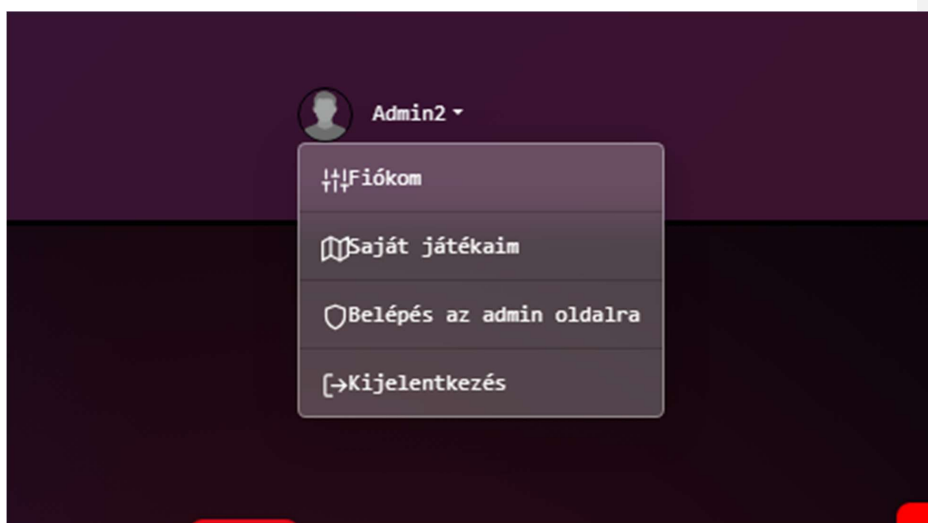


3.4-2-ábra Bejelentkezés

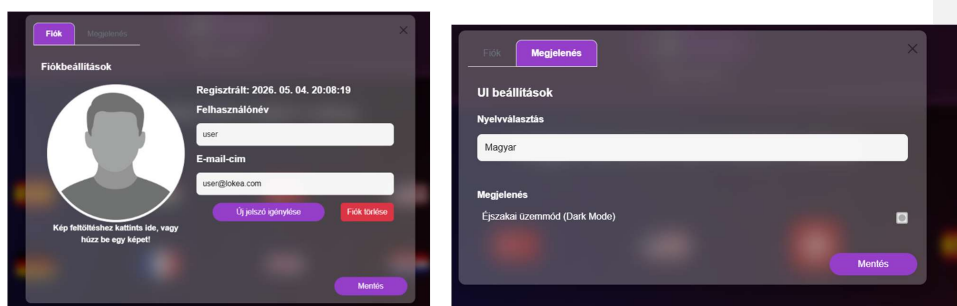
3.4.2 Profilbeállítások

A profilképre húzva az egeret hozzá lehet férni a profilbeállításokhoz, ahol lehet jelszót változtatni, sötétmódot beállítani, nyelvet változtatni.

[NV2] megjegyzést írt: Kép a regisztrációról és a belépésről, beállításokról



3.4.2-1-ábra Profil dropdown



3.4-4. ábra Profil beállítása modalról képernyőkép

3.4.3 Pályák böngészése

A főoldalon a felhasználó áttekintheti a legtöbbet játszott négy pályát. A pályák borítóképpel, címmel, készítővel és átlagos értékeléssel, játékok számával jelennek meg. A főoldaltól a profilra húzva az egeret a saját pályáim fülére kattintva át lehet menni a saját pályák oldalra ahol az új pálya létrehozása is lehetséges. Emellett a főoldaltól egy gomb segítségével lehet átjutni a pályaválasztó oldalra, ahol lehet a pályákat **böngészni**.

[NV3] megjegyzést írt: Kép a játékválasztóról

3.4.4 Pályaoldal

Amint megvan a kívánt pálya, rákattintással át visz a pálya oldalra, ahol lehet kommentelni és megnézni a pályához tartozó top öt pontszámot. Az oldalon lehet beállítani a pálya borítóképét, leírását és nevét, emellett elérhető a pályaváltoztató/csináló oldal is, mindez csak akkor érhető el, ha a felhasználó készítette. Az oldal jobb alsó sarkában található a játékindító gomb **is**.

[NV4] megjegyzést írt: Kép a pályaoldalról

3.4.5 Játékmenet

A játék indításakor a felhasználó testreszabhatja a beállításokat:

- **Körök száma:** Hány panorámaképet kap a játékos.
- **Köridő:** Hány másodperc áll rendelkezésre körönként.

A játék indítása következtében egy visszaszámlálás jön, mely minden kör előtt megtörténik. Utána képernyőn megjelenik a 360 fokos panorámakép (amelyet egérrel vagy érintéssel lehet forgatni), a jobb alsó sarokban pedig az interaktív térkép. A játékos a térképre kattintva jelöli meg a tippjét, majd a „Tipp leadása” gombbal véglegesíti azt.

A kör lezárása után a rendszer megjeleníti a helyes pozíciót, a tipp távolságát pixelben, valamint a kapott pontszámot. A maximálisan kapható pontszám körönként 5000 pont. Az összesített pontszám a ranglista alapja.

[NV5] megjegyzést írt: Kép a játékménetről

3.4.6 Saját pálya készítése

Regisztrált felhasználók a pályaszerkesztő felületen hozhatnak létre saját pályákat. A folyamat lépései:

1. Töltsön fel egy térképet és állítsa be a térkép beállításait, majd mentse el.
2. Töltsön fel pontokat a „+” gomb segítségével, állítsa be a pont északi irányát, fontos, hogy minél pontosabban tegye meg a mozgás miatt, a legjobb játékelményért. Állítson be a pontoknak panorámaképeket, és látni is fogja a feltöltött kép előnézetét.
3. Miután lehelyezett tetszőleges számú pontot össze tudja kötni őket és onnantól kezdve a játékokban létrejön egy nyíl amivel tud köztük mozogni.
4. Ha szeretne több térképet a játékába csak ismételje meg ezeket a lépéseket.
5. Ha minden változtatással végzett, csak lépjen ki az oldalról a navbar segítségével, a rendszer automatikusan menti a változtatásait, és már játszhat is a játékkal.

3.5 Adminisztrációs felület

Az ADMIN és LORD jogosultsággal rendelkező felhasználók számára elérhető az adminisztrációs felület, amely a következő funkciókat biztosítja:

- **Felhasználókezelés:** Felhasználók listázása, keresése, adataik szerkesztése, törlése (soft delete).
- **Statisztikák:** Regisztrációk száma, napi aktivitás, lejátszott meccsek – vizuális grafikonok formájában.
- **Naplózás:** Az összes adminisztrátori beavatkozás visszakövethető a napló (log) felületen.

A LORD szintű felhasználó ezen felül új ADMIN fiókokat is létrehozhat.

3.6 Hibajelzések

Az alkalmazás a felhasználó téves műveleteire egyértelmű hibaüzenetekkel reagál. Az alábbiakban néhány jellemző helyzet:

- **Hibás bejelentkezés:** „Hibás felhasználónév vagy jelszó.” – A rendszer nem árulja el, melyik adat volt hibás, biztonsági okokból.
- **Duplikált felhasználónév vagy e-mail:** „Ez a felhasználónév/e-mail cím már foglalt.”
- **Időtúllépés a játékban:** Ha a kör ideje lejár, a tipp automatikusan 0 ponttal zárul, és a rendszer a következő körre lép.
- **Érvénytelen képfeltöltés:** A rendszer visszautasítja a nem támogatott formátumú vagy túl nagy méretű fájlokat.

4 Csapatmunka megvalósítása

A fejlesztői csapat három főből állt: Eördögh Erik, Pintér Bálint és Varga Norbert. A feladatmegosztás az alkalmazás moduláris felépítését követte, így a tagok párhuzamosan, egymástól nagyrészt függetlenül dolgozhattak.

Csapattag	Feladatkörök
Eördögh Erik	Főoldal frontend- és backend-jének, admin oldal frontend- és backend-jének megtervezése és lekódolása, projekt lokalizációja
Pintér Bálint	Játék motorjának elkészítése, pálya oldal frontend- és backend-jének megtervezése és lekódolása, pálya készítő oldal backend, frontend megcsinálása, központi hibakezelő
Varga Norbert	Játékválasztó és játék oldal frontend- és backend-jének megtervezése és lekódolása, kedvencek megcsinálása frontend, backend

Az együttműködés során a **GitHub** platformot használtuk a verziókezelésre és a feladat nyomon követésére. A fejlesztés során betartottuk az alábbi munkafolyamatot:

- **Branch**-ek használata: Minden új funkció külön fejlesztési ágon (branch) készült, így a főág (main) mindig stabil maradt.
- **Pull request**-ek használata: A kész funkciók beolvasztása előtt a csapat többi tagja átnézte a kódot (code review), biztosítva ezzel a kódminőséget.
- **Issue**-k használata: A feladatokat, hibajelentéseket GitHub issue-k formájában tartottuk nyilván, így a munka átlátható és követhető volt.
- **Rendszeres commitolás**: A fejlesztés teljes folyamata 855 commitban követhető nyomon, amely a folyamatos, párhuzamos munkavégzést tükrözi.

A közös munka során kiemelt figyelmet fordítottunk a folyamatos kapcsolattartásra, így hetente egyszer konferenciát tartottunk a Discord felületén. Ezeken a találkozókön mindenki megosztotta, épp hol tart a feladataival, így mindig tisztán láttuk a projekt haladását, és azt is, ha valakinek segítségre volt szüksége. Továbbá iskolai napokon személyesen is megtekintettük egymás munkáját.

5 Összefoglalás

5.1 Projekt kezdete

A vizsgaprojekt tervezésekor olyan témát kerestünk, amely szakmai kihívást jelent, de közben mindannyian lelkesedni tudunk érte. Választásunk végül egy interaktív, térképalapú alkalmazásra esett, amelynek alapjait a népszerű GeoGuessr játékméchanikája inspirálta.

A célunk nem egy egyszerű másolat elkészítése volt: a játék lényegét – a környezeti elemekből való helyszínfelismerést és a térképes tippelést – megtartottuk, de ezt egy saját, egyedi koncepcióval és extra funkciókkal (saját térképek + saját lokációk feltöltésével) egészítettük ki. Így egy olyan modern webes applikációt hoztunk létre, amely ismerős alapokon nyugszik, mégis újszerű élményt nyújt.

5.2 Projekt során

A hajrában az elemek összeillesztésére és a finomhangolásra koncentráltunk. Ellenőriztük, hogy minden követelménynek megfeleltünk-e, és az esetlegesen felmerülő apróbb hibákat közösen javítottuk. Ez az időszak a tesztelésről és arról szólt, hogy a végeredmény pontosan olyan legyen, amilyennek az elején megálmodtuk.

5.3 Tanulságok és tapasztalatok

A közös munka során nemcsak szakmailag fejlődtünk, hanem sokat tanultunk az időmenedzsmentről és a távoli együttműködésről is. Rájöttünk, hogy a rendszeres visszajelzés mennyire sokat számít a lendület fenntartásában, és arra is rájöttünk, hogy egy projektet sose lehet korán elkezdni. Megtanultuk, hogy a verziókezelés használata elengedhetetlen a káosz elkerüléséhez. Bár adódtak nehézségek, a hatékony kommunikáció és a csapatszellem végül mindig átszegtett minket a holtponatokon.

5.4 A program jövője

A projektmunka után dolgozni fogunk a többszemélyes játékmódok kialakításán, Google Maps API implementálásán (hasonlóan a GeoGuessr-hez).

6 Ábrajegyzék

2.3-1. ábra Az elkészített adatbázis[NV1.1] adatmodellje	4
2.6-1. ábra Pályaaloldalról képernyőkép	14
2.6-2. ábra Játékválasztó oldalról képernyőkép	14
2.6-3. ábra Főoldalról képernyőkép	14
2.6-4. ábra Admin felületről képernyőkép	15
2.6-5. ábra Játékoldalról képernyőkép	15
2.6-6. ábra Pályakészítő oldalról képernyőkép	15
2.8-1. ábra A szótárfájlok (JSON) szerkezete	18
2.9-1 ábra Teszteredmények 1	21
2.9-2- ábra Teszteredmények 2	22
2.9-3-ábra Teszteredmények 3	22
3.4-1. ábra Regisztrációs oldalról képernyőkép	28
3.4-2-ábra Bejelentkezés	29
3.4.2-1-ábra Profil dropdown	29
3.4-4. ábra Profil beállítása modalról képernyőkép	30

7 Irodalomjegyzék

7.1 Weboldalak

- [The Perspective and Orthographic Projection Matrix](#) / (utolsó megtekintés: 2026.05.04)
- [The Pinhole Camera Model](#) / (utolsó megtekintés: 2026.05.04)
- [The Perspective and Orthographic Projection Matrix](#) / (utolsó megtekintés: 2026.05.04)
- Matrices - [Geometry](#) / (utolsó megtekintés: 2026.05.04)
- How does matrix work part 1 - [Geometry](#) / (utolsó megtekintés: 2026.05.04)
- How does matrix work part 2 - [Geometry](#) / (utolsó megtekintés: 2026.05.04)
- Transforming points and vectors - [Geometry](#) / (utolsó megtekintés: 2026.05.04)

- Points and vectors - [Geometry](#) / (utolsó megtekintés: 2026.05.04)
- Node.js dokumentáció – <https://nodejs.org/en/docs/> (utolsó megtekintés: 2026.05.04)
- Express.js dokumentáció – <https://expressjs.com/> (utolsó megtekintés: 2026.05.04)
- MySQL 8.0 Reference Manual – <https://dev.mysql.com/doc/refman/8.0/en/> (utolsó megtekintés: 2026.05.04)
- MDN Web Docs – WebAssembly – <https://developer.mozilla.org/en-US/docs/WebAssembly> (utolsó megtekintés: 2026.05.04)
- Emscripten dokumentáció – <https://emscripten.org/docs/> (utolsó megtekintés: 2026.05.04)
- Sharp (képfeldolgozó könyvtár) dokumentáció – <https://sharp.pixelplumbing.com/> (utolsó megtekintés: 2026.05.04)
- Jest dokumentáció – <https://jestjs.io/docs/getting-started> (utolsó megtekintés: 2026.05.04)
- WebGL Fundamentals – <https://webglfundamentals.org/> (utolsó megtekintés: 2026.05.04)
- Bcrypt – <https://www.npmjs.com/package/bcrypt> (utolsó megtekintés: 2026.05.04)
- Express – validator – <https://express-validator.github.io/docs/guides/validation-chain> (utolsó megtekintés: 2026.05.04)
- i18next - <https://www.i18next.com/overview/getting-started> (utolsó megtekintés: 2026.05.04)
- GitHub Docs – <https://docs.github.com/en> (utolsó megtekintés: 2026.05.04)